NVIDIA NIMs on H200 GPUs

Table of Contents

NVIDIA NIM **4**
Setup 4
Running LLM NIMs on H200 GPUs **5**
llama-3.3-nemotron-super-49b-v1.5 5
Qwen2.5-VL-32B-Instruct 7

Document History

Doc_Number

Version	Date	Authors	Description of Change
01	Mar 5, 2026		Initial release

NVIDIA NIM

NVIDIA Inference Microservices (NIM) are a set of optimized, cloud-native microservices designed to simplify the deployment of generative AI models across various infrastructures, including clouds, data centers, and workstations. NIM abstracts the complexities of AI model development and packaging, enabling developers to deploy models efficiently and maintain control over their workloads. These microservices are prebuilt, optimized containers that support a broad spectrum of AI models, including open-source community models, NVIDIA AI Foundation models, and custom AI models. They are designed to facilitate seamless AI inferencing at scale, ensuring that AI applications can be deployed with confidence on NVIDIA-accelerated infrastructure.

NVIDIA Inference Microservices (NIM) offer two distinct container options for deploying large language models (LLMs): the LLM-specific NIM and the NIM VLM. The LLM-specific NIM container is tailored for individual models or model families, providing maximum performance out-of-the-box. These containers are optimized for specific model and GPU combinations, ensuring that the deployment leverages pre-built, highly optimized engines. The NIM vLLM container, on the other hand, is designed for flexibility and versatility. It supports a broad range of models from Hugging Face. This container is particularly useful when NVIDIA does not yet offer an LLM-specific container for the model you want to deploy.

The following instructions will guide us through downloading the required assets, setting up the environment, and running a NIM.

Useful resources:

- Overview of NVIDIA NIM for Large Language Models (LLMs)
- NIM vLLM

Setup

Before starting, we need to collect the required credentials and define specific environment variables.

We need to obtain two credentials: an NGC API key. To create an NGC API key, go to NVIDIA NGC, click on the “Generate Personal Key button” in the top right, select “NGC Catalog”, "Private Registry", and "Cloud Functions" in the “Services Included” dropdown. Once done, store the API key in a safe location, such as a password manager.

We also need to define a LOCAL_NIM_CACHE environment variable which is needed to select for the LLM-specific NIM:

- `LOCAL_NIM_CACHE="${HOME}/.cache/nim"` defines the location of NIM cache folder which should be mounted into the container,

```shell
export LOCAL_NIM_CACHE="${HOME}/.cache/nim"
mkdir -p "$LOCAL_NIM_CACHE"
chmod 777 "$LOCAL_NIM_CACHE"
```

Running LLM NIMs on H200 GPUs

The following section demonstrates how to deploy two NVIDIA NIMs on a two H200 GPUs for shared inference using the following models:

- **llama-3.3-nemotron-super-49b-v1.5**
- **Qwen2.5-VL-32B-Instruct**

Each NIM is deployed as a separate container and runs on the same GPUs. **llama-3.3-nemotron-super-49b-v1.5** is a reasoning model that is post trained for reasoning, human chat preferences, and agentic tasks, such as RAG and tool calling. **Qwen2.5-VL-32B-Instruct** serves as a high-performance, lightweight vision-language model when compared to the massive scale of Qwen3.5-397b-a17b.

llama-3.3-nemotron-super-49b-v1.5

We first define the NGC API key and log in to NGC.

```shell
export NGC_API_KEY=<insert your NGC KEY HERE>
echo "${NGC_API_KEY}" | docker login nvcr.io -u '$oauthtoken' --password-stdin
```

We run llama-3.3-nemotron-super-49b-v1.5 on a single GPU and expose the service on port 8000.

```shell
docker run -it --rm \
    --gpus '"device=0"' \
    --shm-size=16GB \
    -e NGC_API_KEY \
```

```
    -v "$LOCAL_NIM_CACHE:/opt/nim/.cache" \
    -u $(id -u) \
    -p 8000:8000 \
    nvcr.io/nim/nvidia/llama-3.3-nemotron-super-49b-v1.5:latest
```

The container detects a compatible single-GPU profile and starts downloading it automatically.

```
INFO 2025-12-14 16:29:32.960 profiles.py:343] Detected 1 compatible profile(s).
INFO 2025-12-14 16:29:32.960 profiles.py:354] Matched profile_id in manifest from LLMBasedProfileSelector fc1df044c94b46
6d0ebd561df47556bc23a01ac8147d68dc49f04238a6cfcd7f with tags: {'feat_lora': 'false', 'llm_engine': 'vllm', 'nim_workspac
e_hash_v1': '8d1357e1888e26523f732140e20c1562434517e6f8e5fa12bc9a67bebf202d33', 'pp': '1', 'precision': 'mxfp4', 'tp': '
1'}
```

Once the download is completed and the container launched, the API endpoint is shown in the terminal.

```
[TensorRT-LLM] TensorRT-LLM version: 1.0.3.2510
INFO 2026-03-04 20:08:40.54 dynamic_module_loader.py:101] Loading dynamic modules from /opt/nim/llm/nim_llm_sdk/model_specific_modules
INFO 2026-03-04 20:08:40.127 trtllm_llm_api_metrics.py:103] Stats process worker started
INFO 2026-03-04 20:08:40.128 trtllm_llm_api_metrics.py:303] Started stats process worker (PID: 3627)
Iteration statistics are not available yet. To collect runtime statistics, please call get_stats_async() in async coroutine or the /metrics endpoint (if you're using trtllm
-serve) AFTER prompts have been submitted.
INFO 2026-03-04 20:08:40.166 health.py:59] Health request successful.
INFO 2026-03-04 20:08:40.167 standalone_api_server.py:380] An example cURL request:
curl -X 'POST' \
  'http://0.0.0.0:8000/v1/chat/completions' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "model": "nvidia/llama-3.3-nemotron-super-49b-v1.5",
    "messages": [
      {
        "role":"user",
        "content":"Hello! How are you?"
      },
      {
        "role":"assistant",
        "content":"Hi! I am quite well, how can I help you today?"
      },
      {
        "role":"user",
        "content":"Can you write me a song?"
      }
    ],
    "top_p": 1,
    "n": 1,
    "max_tokens": 15,
    "stream": true,
    "frequency_penalty": 1.0,
    "stop": ["hello"]
  }'
```

The following example shows how to query the running NIM from the terminal:

```shell
 curl -X 'POST' \
'http://0.0.0.0:8000/v1/chat/completions' \
-H 'accept: application/json' \
-H 'Content-Type: application/json' \
-d '{
    "model": "nvidia/llama-3.3-nemotron-super-49b-v1.5",
    "messages": [{"role": "system", "content": "detailed thinking off"},
{"role":"user", "content":"Write a limerick about the wonders of GPU
computing."}],
    "max_tokens": 64
}'
```

We can then inspect GPU usage, using *nvidia-smi*:

```
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 550.163.01             Driver Version: 550.163.01     CUDA Version: 12.4     |
|-----------------------------------------+------------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                         |                        |               MIG M. |
|=========================================+========================+======================|
|   0  NVIDIA H200                    On  |   00000000:8D:00.0 Off |                    0 |
| N/A   31C    P0            117W /  700W |  130915MiB / 143771MiB |      0%      Default |
|                                         |                        |             Disabled |
+-----------------------------------------+------------------------+----------------------+
```

It can also be queried from a Python IDE using the local endpoint, following the same structure as the OpenAI API:

```python
from openai import OpenAI

client = OpenAI( base_url="http://localhost:8000/v1", api_key="EMPTY")

completion = client.chat.completions.create(
  model="nvidia/llama-3.3-nemotron-super-49b-v1.5",
  messages=[{"role":"system","content":"/think"}],
  temperature=0.6,
  top_p=0.95,
  max_tokens=6553,
  frequency_penalty=0,
  presence_penalty=0,
  stream=True
)
```

Qwen2.5-VL-32B-Instruct

We then deploy the second vision language model. A different GPU and port are used. We will need to define HF_token to download the weights of the model.

```shell
# Set your token
export HF_TOKEN="your_huggingface_token_here"

docker run --runtime nvidia --gpus '"device=1"' \
    -u root \
    -v /var/tmp/hf_cache:/data \
    -e HF_HOME=/data \
```

```
    -e HUGGING_FACE_HUB_TOKEN=$HF_TOKEN \
    -p 8001:8000 \
    --ipc=host \
    nvcr.io/nvidia/vllm:26.01-py3 \
    vllm serve Qwen/Qwen2.5-VL-32B-Instruct \
    --dtype bfloat16 \
    --max-model-len 8192 \
    --limit-mm-per-prompt '{"image": 5, "video": 0}' \
    --trust-remote-code
```

Similarly, once launched, it is shown in the terminal

```
(APIServer pid=1) INFO 03-04 18:46:01 [launcher.py:46] Route: /rerank, Methods: POST
(APIServer pid=1) INFO 03-04 18:46:01 [launcher.py:46] Route: /v1/rerank, Methods: POST
(APIServer pid=1) INFO 03-04 18:46:01 [launcher.py:46] Route: /v2/rerank, Methods: POST
(APIServer pid=1) INFO 03-04 18:46:01 [launcher.py:46] Route: /pooling, Methods: POST
(APIServer pid=1) INFO:     Started server process [1]
(APIServer pid=1) INFO:     Waiting for application startup.
(APIServer pid=1) INFO:     Application startup complete.
```

Once the endpoint is active, we can query it through any Python IDE. Since the NIM is running locally on your H200 node, a valid API key is not required for authentication.

```python
import base64
from openai import OpenAI

# 1. Setup - Points to your VS Code tunnel
client = OpenAI(
    base_url="http://localhost:8001/v1",
    api_key="EMPTY"
)

# 2. Your specific Windows path
IMAGE_PATH = "id_example.png"

def encode_image(path):
    """Encodes a local image file to a base64 string."""
    with open(path, "rb") as image_file:
        return base64.b64encode(image_file.read()).decode('utf-8')

# 3. Create and send the request
try:
    print(f"Encoding image from: {IMAGE_PATH}")
    base64_image = encode_image(IMAGE_PATH)
```

```
    response = client.chat.completions.create(
        model="Qwen/Qwen2.5-VL-32B-Instruct",
        messages=[
            {
                "role": "user",
                "content": [
                    {"type": "text", "text": "Extract text in the image."},
                    {
                        "type": "image_url",
                        "image_url": {
                            "url": f"{base64_image}"
                        },
                    },
                ],
            }
        ],
        max_tokens=300,
    )

    print("\n--- Model Response ---")
    print(response.choices[0].message.content)

except Exception as e:
    print(f"\nError: {e}")
```